

PKI - Plateforme d'identité numérique

Intégration à l'architecture Cloud : Consul + API Gateway + Microservices — Guide pour les équipes

*Enregistrement & découverte de services · Health-checks · Sécurité
OAuth2 via Keycloak · Spring Boot · Quarkus · Laravel · Python · Node.js*





Version	Date	Auteurs	Description
1.1	22/06/2026	Judicaël ZANNOU	Initialisation de la Documentation
1.2	24/06/2026	Judicaël ZANNOU	• Réorganisation par composant (API Gateway, Consul, Keycloak) ; • ajout des sections API Gateway et Keycloak ; • renumérotation et mise à jour de la table des matières.



Table des matières

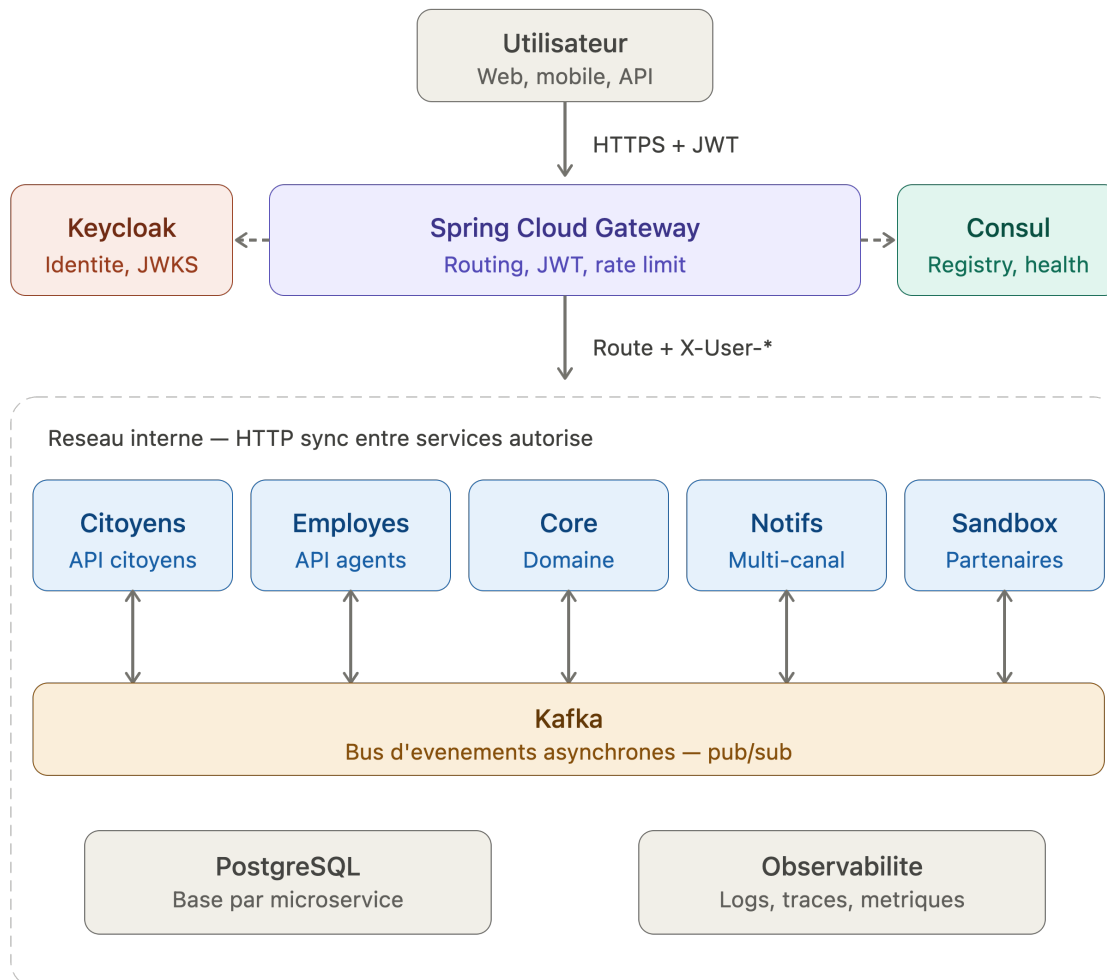
INTRODUCTION	4
1 API GATEWAY	5
2 CONSUL	6
2.1 CONCEPTS CONSUL ESSENTIELS	6
2.2 URLS DE BASE ET ENVIRONNEMENTS	6
2.3 CONVENTIONS D'INTÉGRATION ASIN	6
2.4 SECURITE : MODELE D'IDENTITE KEYCLOAK	7
2.5 OBTENIR UNE IDENTITE DE SERVICE (TOKEN)	7
2.5.1 ÉTAPE 1 — TOKEN KEYCLOAK (CLIENT_CREDENTIALS)	7
2.5.2 ÉTAPE 2 — ECHANGE CONTRE UN TOKEN ACL CONSUL	7
2.6 INTEGRATION — SPRING BOOT	7
2.6.1 DÉPENDANCES (POM.XML)	7
2.6.2 CONFIGURATION (APPLICATION.YAML)	8
2.6.3 DECOUVERTE COTE CLIENT	8
2.7 INTEGRATION — QUARKUS (JAVA)	8
2.7.1 EXTENSIONS (POM.XML)	8
2.7.2 DÉCOUVERTE (APPLICATION.PROPERTIES)	8
2.7.3 ENREGISTREMENT AU DEMARRAGE (AVEC TOKEN + HTTPS)	8
2.8 INTEGRATION — PHP / LARAVEL	9
2.8.1 ROUTE DE SANTE (ROUTES/API.PHP)	9
2.8.2 COMMANDE D'ENREGISTREMENT (AVEC TOKEN)	9
2.9 INTEGRATION — PYTHON (FLASK & DJANGO)	10
2.9.1 HELPER COMMUN (CONSUL_REGISTRY.PY)	10
2.9.2 FLASK	10
2.9.3 DJANGO	10
2.10 INTÉGRATION — NODE.JS	11
2.11 ENREGISTREMENT PAR FICHIER DE DEFINITION (UNIVERSEL)	11
2.12 DECOUVRIR LES AUTRES SERVICES (DNS & API)	12
2.12.1 VIA DNS (TOUT LANGAGE)	12
2.12.2 VIA L'API HTTP (INSTANCES SAINES)	12
2.13 VERIFICATION ET DEPANNAGE	12
2.14 CHECKLIST D'INTÉGRATION	12
3 KEYCLOAK	13
3.1 LE MICROSERVICE ET KEYCLOAK : DEUX ROLES	13
3.2 VALIDER UN JETON ENTRANT	13
3.3 MISE EN ŒUVRE PAR FRAMEWORK	13
4 ANNEXE — VARIABLES D'ENVIRONNEMENT	13



Introduction

Ce document décrit l'intégration d'un microservice à l'architecture cloud de la plateforme MOD-ID de l'ASIN. Cette architecture repose sur quatre piliers complémentaires — une API Gateway en façade, l'annuaire de services Consul, le fournisseur d'identité Keycloak et un ensemble de microservices métier — auxquels s'ajoute le bus d'événements Kafka pour les échanges asynchrones. Le schéma ci-dessous en présente une vue d'ensemble.

Ce document s'adresse aux équipes qui développent un microservice destiné à s'intégrer à l'architecture MOD-ID de l'ASIN, quel que soit le langage. Il explique comment enregistrer votre service dans l'annuaire Consul, exposer un point de santé, découvrir les autres services, et sécuriser ces échanges via Keycloak.



Legende





L'écosystème se compose des composants suivants :

- **API Gateway — Porte d'entrée.** Elle ne stocke rien et ne calcule aucune logique métier. Elle route les requêtes vers le bon service, vérifie les tokens, applique des limites (rate limiting, quotas) et propage les informations utiles (identité, traçabilité) vers les services en aval.
- **Consul — Annuaire téléphonique des services.** Il sait qui est en vie, où chaque service écoute et s'il répond correctement (health-checks). Il stocke aussi des configurations partagées via son KV store. C'est le cœur de la découverte de services détaillée dans la suite de ce document.
- **Keycloak — Bureau des passeports.** Après avoir authentifié l'appelant, il émet des tokens JWT signés cryptographiquement. Il publie sa clé publique (JWKS) afin que les autres composants puissent vérifier les signatures sans avoir à le recontacter.
- **Microservices — Les vrais travailleurs.** Chacun porte une responsabilité métier précise (Citoyens, Employés, Core, etc.) et expose une API HTTP. C'est l'un de ces services que vous allez enregistrer dans Consul.
- **Kafka — Service postal interne (bus d'événements).** Il achemine les messages entre services de façon asynchrone et découplée : un service publie un événement (« citoyen créé », « document validé »), et les services intéressés le consomment à leur rythme. Les messages sont persistés et peuvent être rejoués, ce qui évite que les services s'attendent mutuellement et absorbe les pics de charge.

Règle de communication : Kafka est utilisé pour les échanges asynchrones (événements, traitements différés). Les appels HTTP synchrones directs entre services restent autorisés, notamment pour les besoins de lecture immédiate.

La suite de ce document se concentre sur l'intégration à Consul — point d'entrée de tout microservice dans l'écosystème : enregistrement dans l'annuaire, exposition d'un point de santé, découverte des autres services et sécurisation des échanges via Keycloak.

1 API Gateway

L'API Gateway est le point d'entrée unique de la plateforme : c'est le seul composant exposé directement à Internet. Tout le trafic externe transite par elle avant d'atteindre un microservice. Elle ne porte aucune logique métier ; elle concentre les préoccupations transverses, que l'on peut regrouper en six grandes catégories :

- **Routage et résolution dynamique** — elle détermine, pour chaque requête, le service cible et achemine l'appel vers une instance saine (résolution via Consul).
- **Authentification et autorisation centralisées** — elle vérifie le token JWT émis par Keycloak et applique les règles d'accès en amont, avant que la requête n'atteigne un service.
- **Propagation du contexte d'identité** — elle extrait l'identité de l'appelant depuis le token et la transmet aux services en aval (en-têtes), pour qu'ils n'aient pas à ré-authentifier.
- **Protection et résilience** — rate limiting, quotas, time-outs, retries et circuit breakers protègent les services contre les surcharges et les pannes en cascade.
- **Observabilité** — premier point de mesure du trafic entrant : journaux, métriques et traces centralisés.
- **Préoccupations transverses HTTP** — terminaison TLS, CORS, compression et normalisation des en-têtes.



2 Consul

La plateforme utilise Consul (HashiCorp) comme registre de services. Consul est agnostique du langage : tout service capable de faire des requêtes HTTP peut s'y enregistrer.

2.1 Concepts Consul essentiels

Service : une entrée d'annuaire — nom logique, adresse IP, port, tags, health-check.

Agent : process Consul local (sur chaque hôte) ; vos services lui parlent sur le port 8500/8501. En test ou production, ne jamais contacter les serveurs directement.

Health-check : l'agent appelle périodiquement un endpoint HTTP de votre service ; s'il échoue, le service est retiré de l'annuaire.

Découverte : trouver une instance saine via l'API (`/v1/health/service/<nom>?passing`) ou via DNS (`<nom>.service.consul`).

2.2 URLs de base et environnements

Ressource	Développement	Test
API Consul	http://localhost:8500 (HTTP)	https://IP:PORT (HTTPS, port 8501)
UI Consul	http://localhost:8500/ui	https://IP:PORT/ui
DNS Consul	localhost:8600 (UDP)	résolveur interne :8600
Keycloak (realm infra)	https://test-oauth.identite-numerique.bj/realms/infra	https://test-oauth.identite-numerique.bj/realms/infra
Endpoint Keycloak token	https://test-oauth.identite-numerique.bj/realms/infra/protocol/openid-connect/token	idem
JWKS Keycloak	https://test-oauth.identite-numerique.bj/realms/infra/protocol/openid-connect/certs	idem

Note : en développement local, Consul tourne en mode -dev (sans TLS mais avec ACL). En test ou production, l'API est en HTTPS (8501) et protégée par ACL : tous les appels portent un token.

2.3 Conventions d'intégration ASIN

Convention	Règle
Nom du service	kebab-case ; préfixe « portal-id- » pour les services cœur. Identique au nom DNS, à l'ID des routes gateway, et au clientId Keycloak (§2.4).
Endpoint de santé	Spring : /actuator/health. Autres langages : /health. Renvoie 200 si sain.
Intervalle de check	10s.
Adresse enregistrée	IP joignable (pas un hostname VPN).
instance-id	<nom>-<aléatoire> (autorise plusieurs instances).
Tags	« asin », version d'API (« v1 »), profil (« prod »).
Désenregistrement	à l'arrêt propre + deregister_critical_service_after côté check.

Routage gateway : la Spring Cloud Gateway résout « `lb://<nom>` » via Consul. Le nom enregistré doit correspondre EXACTEMENT à l'ID de route.



2.4 Sécurité : modèle d'identité Keycloak

En test ou en production, l'API Consul est protégée par le système ACL : chaque appel doit porter un token. Ces tokens sont délivrés par Keycloak via un realm dédié à l'infrastructure (« infra »), séparé des realms utilisateurs.

Chaque service dispose d'un client Keycloak confidentiel (grant client_credentials) dans le realm infra. Le clientId est le NOM EXACT du service (ex. portal-id-employee). Le service obtient un token Keycloak, l'échange contre un token ACL Consul, puis l'utilise pour s'enregistrer et découvrir.

Couche	Assurée par Keycloak ?
Identité du service / autorisation d'enregistrement (ACL)	✓ Oui
Accès opérateur à l'UI (login OIDC)	✓ Oui
Chiffrement du transport (TLS HTTPS/RPC)	✗ Non — certificats (PKI)
Chiffrement inter-agents (gossip)	✗ Non — clé symétrique

Important : Keycloak gère l'identité, pas le chiffrement. Le TLS (HTTPS sur 8501) et la clé gossip sont activés séparément par l'équipe infrastructure PKI-ASIN.

2.5 Obtenir une identité de service (token)

Étapes communes à tous les langages. L'équipe ASIN vous fournit : le clientId (= nom du service) et le client secret du realm infra.

2.5.1 Étape 1 — token Keycloak (client_credentials)

```
curl -s -X POST \
  https://test-oauth.identite-numerique.bj/realms/infra/protocol/openid-connect/token \
  -d grant_type=client_credentials \
  -d client_id=portal-id-exemple \
  -d client_secret=$KC_INFRA_SECRET
# => { "access_token": "eyJ...", "expires_in": 300, ... }
```

2.5.2 Étape 2 — échange contre un token ACL Consul

```
curl -s -X POST https://<IP-CONSUL:PORT>/v1/acl/login \
  --cacert /etc/consul/tls/ca.pem \
  -d '{"AuthMethod":"keycloak-infra-svc","BearerToken":"'$ACCESS_TOKEN'"'}'
# => { "SecretID": "<token-ACL-consul>", ... }
```

Ce token ACL est ensuite passé dans l'en-tête X-Consul-Token (ou la variable CONSUL_HTTP_TOKEN) de chaque appel. En développement local (mode -dev), aucune de ces étapes n'est nécessaire.

2.6 Intégration — Spring Boot

Stratégie native (starter). Standard de la plateforme.

2.6.1 Dépendances (pom.xml)

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-consul-discovery</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
```



</dependency>

2.6.2 Configuration (application.yaml)

```
spring:
  application:
    name: portal-id-exemple    # = nom du service = clientId Keycloak
  cloud:
    consul:
      host: ${CONSUL_HOST:localhost}
      port: ${CONSUL_PORT:8500}
      scheme: ${CONSUL_SCHEME:http}    # https en production
      token: ${CONSUL_HTTP_TOKEN;}    # token ACL (prod)
      discovery:
        prefer-ip-address: true
        health-check-path: /actuator/health
        health-check-interval: 10s
        instance-id: ${spring.application.name}-${random.value}
        tags: [asin, v1]
```

Token : spring-cloud-consul attend un token ACL déjà obtenu (il ne fait pas l'échange OIDC lui-même). Récupérez-le au démarrage (composant d'init appellant /v1/acl/login) et injectez-le via CONSUL_HTTP_TOKEN.

2.6.3 Découverte côté client

Via la gateway : route « uri: lb://portal-id-exemple ». En interne : DiscoveryClient ou un RestClient/WebClient @LoadBalanced.

2.7 Intégration — Quarkus (Java)

Découverte via SmallRye Stork ; enregistrement via l'API de l'agent au démarrage ; santé via /q/health.

2.7.1 Extensions (pom.xml)

```
<dependency><groupId>io.quarkus</groupId>
<artifactId>quarkus-smallrye-health</artifactId></dependency>
<dependency><groupId>io.quarkus</groupId>
<artifactId>quarkus-smallrye-stork</artifactId></dependency>
<dependency><groupId>io.smallrye.stork</groupId>
<artifactId>stork-service-discovery-consul</artifactId></dependency>
<dependency><groupId>io.quarkus</groupId>
<artifactId>quarkus-rest-client</artifactId></dependency>
```

2.7.2 Découverte (application.properties)

```
quarkus.stork.portal-id-employee.service-discovery.type=consul
quarkus.stork.portal-id-employee.service-discovery.consul-host=${CONSUL_HOST:localhost}
quarkus.stork.portal-id-employee.service-discovery.consul-port=${CONSUL_PORT:8500}
quarkus.stork.portal-id-employee.load-balancer.type=round-robin
```

2.7.3 Enregistrement au démarrage (avec token + HTTPS)

```
@ApplicationScoped
public class ConsulRegistration {
    @ConfigProperty(name="quarkus.application.name") String name;
    @ConfigProperty(name="quarkus.http.port") int port;
    @ConfigProperty(name="CONSUL_URL", defaultValue="http://localhost:8500") String consulUrl;
    @ConfigProperty(name="CONSUL_HTTP_TOKEN", defaultValue="") String token;
    @Inject Vertx vertx; private WebClient http; private String id;

    void onStart(@Observes StartupEvent ev) {
        http = WebClient.create(vertx);
```



```
id = name + "-" + UUID.randomUUID();
String ip = "127.0.0.1"; // résoudre l'IP locale réelle
JsonObject body = new JsonObject()
    .put("ID", id).put("Name", name).put("Address", ip).put("Port", port)
    .put("Tags", new JsonArray().add("asin").add("v1"))
    .put("Check", new JsonObject()
        .put("HTTP", "http://" + ip + ":" + port + "/q/health")
        .put("Interval", "10s")
        .put("DeregisterCriticalServiceAfter", "1m"));
http.putAbs(consulUrl + "/v1/agent/service/register")
    .putHeader("X-Consul-Token", token).sendJsonObject(body);
}
void onStop(@Observes ShutdownEvent ev) {
    http.putAbs(consulUrl + "/v1/agent/service/deregister/" + id)
        .putHeader("X-Consul-Token", token).send();
}
}
```

2.8 Intégration — PHP / Laravel

Enregistrement via l'API HTTP Consul, déclenché par une commande Artisan au déploiement, plus une route de santé.

2.8.1 Route de santé (routes/api.php)

```
Route::get('/health', function () {
    try { \DB::connection()->getPdo(); }
    catch (\Throwable $e) {
        return response()->json(['status'=>'DOWN'], 503);
    }
    return response()->json(['status'=>'UP'], 200);
});
```

2.8.2 Commande d'enregistrement (avec token)

```
class ConsulRegister extends Command {
    protected $signature = 'consul:register';
    public function handle() {
        $name = env('SERVICE_NAME', 'portal-id-php');
        $ip = env('SERVICE_IP', gethostname(gethostname()));
        $port = (int) env('SERVICE_PORT', 8080);
        $id = $name.'-'.\Str::random(8);
        // token ACL Consul (déjà obtenu via Keycloak, cf. §2.5)
        $token = env('CONSUL_HTTP_TOKEN', '');
        $base = env('CONSUL_URL', 'http://localhost:8500');
        \Http::withHeaders(['X-Consul-Token' => $token])
            ->put("$base/v1/agent/service/register", [
                'ID'=>$id, 'Name'=>$name, 'Address'=>$ip, 'Port'=>$port,
                'Tags'=>['asin','v1','laravel'],
                'Check'=>['HTTP'=>"http://$ip:$port/api/health",
                    'Interval'=>'10s','Timeout'=>'2s',
                    'DeregisterCriticalServiceAfter'=>'1m'],
            ]);
        $this->info("Enregistré: $id");
    }
}
```

Au déploiement (entrypoint Docker / supervisor) : `php artisan consul:register`. Pour récupérer le token Keycloak en PHP : un appel `\Http::asForm()->post()` vers l'endpoint token (§2.5.1), puis `\Http::post()` vers `/v1/acl/login`.



2.9 Intégration — Python (Flask & Django)

Bibliothèque python-consul2.

```
pip install python-consul2 requests
```

2.9.1 Helper commun (consul_registry.py)

```
import os, socket, uuid, atexit, consul, requests

def _consul_token():
    """Keycloak client_credentials -> token ACL Consul (prod)."""
    if os.getenv('CONSUL_HTTP_TOKEN'):
        return os.environ['CONSUL_HTTP_TOKEN']
    kc = requests.post(os.environ['KC_INFRA_TOKEN_URI'], data={
        'grant_type': 'client_credentials',
        'client_id': os.environ['KC_INFRA_CLIENT_ID'],
        'client_secret': os.environ['KC_INFRA_SECRET']}).json()['access_token']
    r = requests.post(f'{os.environ['CONSUL_URL']}/v1/acl/login",
        json={'AuthMethod': 'keycloak-infra-svc', 'BearerToken': kc})
    return r.json()['SecretID']

def register(name, port, health_path='/health'):
    token = _consul_token() if os.getenv('CONSUL_URL') else None
    c = consul.Consul(host=os.getenv('CONSUL_HOST', 'localhost'),
        port=int(os.getenv('CONSUL_PORT', 8500)),
        scheme=os.getenv('CONSUL_SCHEME', 'http'), token=token)
    ip = socket.gethostbyname(socket.gethostname())
    sid = f'{name}-{uuid.uuid4().hex[:8]}'
    c.agent.service.register(name=name, service_id=sid, address=ip, port=port,
        tags=['asin', 'v1', 'python'],
        check=consul.Check.http(f'http://{ip}:{port}{health_path}',
            interval='10s', timeout='2s', deregister='1m'))
    atexit.register(lambda: c.agent.service.deregister(sid))
    return sid
```

2.9.2 Flask

```
from flask import Flask, jsonify
from consul_registry import register
app = Flask(__name__)

@app.get('/health')
def health(): return jsonify(status='UP'), 200

if __name__ == '__main__':
    register('portal-id-flask', 5000)
    app.run(host='0.0.0.0', port=5000)
```

2.9.3 Django

```
# views.py
from django.http import JsonResponse
def health(request): return JsonResponse({'status': 'UP'}, status=200)
# urls.py -> path('health', health)

# apps.py
from django.apps import AppConfig
import os
class CoreConfig(AppConfig):
    name = 'core'
    def ready(self):
```



```
if os.environ.get('RUN_MAIN') != 'true':
    return # éviter le double-enregistrement de l'autoreloader
from consul_registry import register
register('portal-id-django', int(os.getenv('SERVICE_PORT', 8000)))
```

2.10 Intégration — Node.js

npm install consul express

```
const express = require('express');
const Consul = require('consul');
const os = require('os'); const crypto = require('crypto');

const app = express();
const PORT = parseInt(process.env.SERVICE_PORT || '3000', 10);
const NAME = process.env.SERVICE_NAME || 'portal-id-node';
const ip = Object.values(os.networkInterfaces()).flat()
    .find(i => i.family === 'IPv4' && !i.internal)?.address || '127.0.0.1';
const id = `${NAME}-${crypto.randomBytes(4).toString('hex')}`;

const consul = new Consul({
    host: process.env.CONSUL_HOST || 'localhost',
    port: process.env.CONSUL_PORT || '8500',
    secure: (process.env.CONSUL_SCHEME === 'https'),
    defaults: { token: process.env.CONSUL_HTTP_TOKEN || undefined },
});

app.get('/health', (req, res) => res.status(200).json({ status: 'UP' }));

app.listen(PORT, async () => {
    await consul.agent.service.register({
        id, name: NAME, address: ip, port: PORT, tags: ['asin', 'v1', 'node'],
        check: { http: `http://${ip}:${PORT}/health`, interval: '10s',
            timeout: '2s', deregistercriticalserviceafter: '1m' },
    });
    console.log(` Enregistré dans Consul: ${id} `);
});

for (const s of ['SIGINT', 'SIGTERM'])
    process.on(s, async () => { await consul.agent.service.deregister(id); process.exit(0); });
```

2.11 Enregistrement par fichier de définition (universel)

Sans aucun code : déposer un JSON auprès de l'agent, puis recharger. Idéal pour un service tiers ou un enregistrement piloté par le déploiement (CI/CD, Ansible).

```
# /etc/consul.d/portal-id-exemple.json
{
  "service": {
    "name": "portal-id-exemple",
    "address": "10.10.12.60", "port": 8080,
    "tags": ["asin", "v1"],
    "check": {
      "http": "http://10.10.12.60:8080/health",
      "interval": "10s", "timeout": "2s",
      "deregister_critical_service_after": "1m"
    }
  }
}
```



consul reload

2.12 Découvrir les autres services (DNS & API)

2.12.1 Via DNS (tout langage)

```
curl http://portal-id-employee.service.consul:8081/api/...
dig @localhost -p 8600 portal-id-employee.service.consul SRV
```

2.12.2 Via l'API HTTP (instances saines)

```
curl -H "X-Consul-Token: $CONSUL_HTTP_TOKEN" \
https://<CONSUL_URL>/v1/health/service/portal-id-employee?passing
```

2.13 Vérification et dépannage

```
# Présence
curl -H "X-Consul-Token: $T" https://<CONSUL_URL>/v1/catalog/service/<nom>
# Santé
curl -H "X-Consul-Token: $T" https://<CONSUL_URL>/v1/health/service/<nom>?passing
# UI
https://<CONSUL_URL>/ui
```

Symptôme	Cause probable	Correctif
Service absent	Enregistrement non exécuté / mauvaise URL / token manquant	Vérifier logs, CONSUL_URL et CONSUL_HTTP_TOKEN.
Présent mais « critical »	Health-check échoue (404/503)	/health (ou /actuator/health) doit répondre 200 sur l'IP/port enregistrés.
403 Permission denied	Token ACL absent/insuffisant	Vérifier le token et le rôle consul-service côté Keycloak.
Jamais retiré après crash	deregister manquant	Ajouter <code>deregister_critical_service_after</code> .
Gateway ne route plus (lb://)	Nom ≠ ID de route	Aligner les noms.

2.14 Checklist d'intégration

- ☐ Endpoint de santé HTTP renvoyant 200 quand le service est sain.
- ☐ Nom de service conforme (kebab-case, = clientId Keycloak).
- ☐ Client Keycloak (realm infra) obtenu auprès de l'équipe ASIN (clientId + secret).
- ☐ Token Keycloak -> token ACL Consul récupéré au démarrage (prod).
- ☐ CONSUL_URL/HOST/PORT/SCHEME et CONSUL_HTTP_TOKEN lus depuis l'environnement.
- ☐ Enregistrement au démarrage + désenregistrement à l'arrêt.
- ☐ Health-check : interval 10s, deregister 1m.
- ☐ IP joignable enregistrée ; instance-id unique ; tags asin/v1.
- ☐ Vérifié sain dans l'UI et via /v1/health/service/<nom>?passing.
- ☐ Si exposé par la gateway : route lb://<nom> alignée.



3 Keycloak

Keycloak est le fournisseur d'identité (IdP) de la plateforme : il authentifie les appelants et émet des jetons JWT signés cryptographiquement. Cette section décrit l'intégration côté microservice — comment un service valide les jetons qu'il reçoit et exploite l'identité qu'ils portent. L'obtention d'un jeton de service pour s'enregistrer dans Consul est, elle, traitée au §2.5.

3.1 Le microservice et Keycloak : deux rôles

Un microservice interagit avec Keycloak de deux manières complémentaires :

- **Serveur de ressources (resource server)** — il reçoit des requêtes, transmises par l'API Gateway, porteuses d'un jeton JWT. Il doit valider ce jeton et en extraire l'identité avant de servir la réponse.
- **Client machine-to-machine** — pour appeler lui-même un autre service ou l'API Consul, il obtient son propre jeton via le flux client_credentials du realm infra (voir §2.5).

3.2 Valider un jeton entrant

La validation repose sur la signature cryptographique du jeton : elle ne nécessite aucun appel synchrone à Keycloak à chaque requête.

- Récupérer la clé publique de signature via l'endpoint JWKS du realm (mise en cache, rafraîchie périodiquement).
- Vérifier la signature, l'émetteur (iss), l'audience (aud) et la date d'expiration (exp) du jeton.
- Extraire les rôles et claims utiles (sub, realm_access.roles, etc.) pour décider de l'autorisation.

Défense en profondeur : même si l'API Gateway a déjà contrôlé le jeton, chaque microservice le revalide (principe zero-trust).

3.3 Mise en œuvre par framework

Les principaux frameworks disposent d'un support standard pour valider les jetons Keycloak :

Framework	Mécanisme
Spring Boot	Spring Security — « OAuth2 Resource Server » (jwt) ; propriétés issuer-uri / jwk-set-uri.
Quarkus (Java)	Extension quarkus-oidc configurée en mode service (resource server).
PHP / Laravel	Middleware de validation JWT (signature via JWKS) ou package OIDC.
Python (Flask & Django)	Bibliothèque python-jose ou authlib : vérification de la signature via JWKS.
Node.js	Middlewares express-jwt + jwks-rsa.

4 Annexe — variables d'environnement

```
# Consul
CONSUL_HOST=localhost      # ou l'hôte de l'agent
CONSUL_PORT=8500           # 8501 en HTTPS prod
CONSUL_SCHEME=http         # https en prod
```



```
CONSUL_URL=http://localhost:8500 # base API complète
CONSUL_HTTP_TOKEN= # token ACL (prod)
CONSUL_CACERT=/etc/consul/tls/ca.pem

# Keycloak (realm infra) — fournis par l'équipe ASIN
KC_INFRA_TOKEN_URI=https://test-oauth.identite-numerique.bj/realms/infra/protocol/openid-connect/token
KC_INFRA_JWKS=https://test-oauth.identite-numerique.bj/realms/infra/protocol/openid-connect/certs
KC_INFRA_CLIENT_ID=portal-id-exemple
KC_INFRA_SECRET=<secret-coffré>

# Service
SERVICE_NAME=portal-id-exemple
SERVICE_PORT=8080
```